SPRAWOZDANIE Z LABORATORIUM 4

Temat: Programowanie po stronie serwera – PHP 1 (Formularze, Walidacja, Pliki)
Przedmiot: Podstawy Technologii Internetowych / Aplikacje Internetowe

Kierunek / Semestr / Grupa:	Informatyka / Semestr 5 / 3ID11B (Podgrupa 06)
Student 1 (Autor):	Artem Hizhovskyi (nr albumu: s097882)
Student 2 (Autor):	Ihor Namachynskyi (nr albumu: s097889)
Prowadzący zajęcia:	Dr inż. Jarosław Wikarek
Data wykonania ćwiczenia:	Październik 2026 r.

1. Formularz HTML ze wszystkimi wymaganymi polami

Zaprojektowano stronę a1.php realizującą kompletny formularz rejestracyjny zgodnie z makietą wykładowcy:

- Nazwisko: pole tekstowe (długość 2-24 znaki, polskie litery, jedno- lub dwuczłonowe),

- Typ zajęć: lista rozwijana select (stacjonarnie, zdalnie),

- Język: przełączniki radio (Polski, Angielski),

- Data urodzenia: pole date,

- PESEL: pole tekstowe 11 cyfr,

- Kursy: pola wielokrotnego wyboru checkbox (Programowanie, Muzyka, Taniec, Gimnastyka).

W formularzu zaimplementowano zapamiętywanie wprowadzonych wartości (sticky form) w przypadku wystąpienia błędów walidacji.

--- [MIEJSCE NA ZRZUT EKRANOWY DLA: 1. Formularz HTML rejestracji] ---

2. Rygorystyczna walidacja danych wyłącznie po stronie PHP

Zgodnie z wymaganiami cała walidacja odbywa się po stronie serwera PHP:

1. Nazwisko: sprawdzanie typów (is_string), całkowita długość mb_strlen od 2 do 24 znaków, wyrażenie regularne dopuszczające wyłącznie litery alfabetu polskiego i jedno- lub dwuczłonowy format.

2. Typ zajęć i Język: walidacja typu 'biała lista' (in_array) uniemożliwiająca podstawienie nieoczekiwanych wartości.

3. Kursy: wymóg zaznaczenia co najmniej jednego kursu, weryfikacja czy nadesłana wartość jest tablicą oraz filtrowanie identyfikatorów względem białej listy.

4. Kalendarzowa walidacja daty urodzenia i wieku: funkcja checkdate() odrzuca nieistniejące kombinacje (np. 31 lutego). Sprawdzenie wymogu wieku min. 16 lat za pomocą DateTime.

5. Algorytm weryfikacji numeru PESEL: 11 cyfr (ctype_digit), dekodowanie stulecia (1800-2200 z uwzględnieniem modyfikacji miesiąca +20, +40, +60, +80) oraz rygorystyczne porównanie z podaną datą urodzenia.

Przy każdej nowej próbie wysłania formularza poprzedni stan sesji jest natychmiast czyszczony (unset($_SESSION['validated_order'])).

```
$nazwisko_len = mb_strlen($nazwisko, 'UTF-8');
if (empty($nazwisko)) {
    $errors['nazwisko'] = "* Pole nie może być puste;";
} elseif ($nazwisko_len < 2 || $nazwisko_len > 24) {
    $errors['nazwisko'] = "* Nazwisko musi zawierać od 2 do 24 znaków;";
} elseif (!preg_match('/^[a-zA-ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+( [a-zA-
ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+)?$/u', $nazwisko)) {
    $errors['nazwisko'] = "* Tylko litery (dozwolone nazwisko jedno- lub
dwuczłonowe);";
}
```

--- [MIEJSCE NA ZRZUT EKRANOWY DLA: 2. Walidacja danych wyłącznie po stronie PHP] ---

3. Dwuetapowy mechanizm potwierdzenia zapisu (Zapisz / Anuluj)

Po bezbłędnej walidacji formularz przechodzi w tryb podsumowania danych, wyświetlając użytkownikowi zestawienie wprowadzonych informacji oraz przyciski «Zapisz» i «Anuluj».

Zabezpieczenie przed ominięciem walidacji: zweryfikowane dane przechowywane są w sesji serwera ($_SESSION['validated_order']). W momencie zatwierdzenia zapisu skrypt odczytuje dane wyłącznie z sesji, uniemożliwiając sfałszowanie parametrów zapytania POST.

```
// Po udanej walidacji:
$_SESSION['validated_order'] = [
    'nazwisko' => $nazwisko,
    'typ_zajec' => $typ_zajec,
    'jezyk' => $jezyk,
    'data_urodzenia' => $data_urodzenia,
    'pesel' => $pesel,
    'kursy' => $kursy
];
$show_confirmation = true;
```

--- [MIEJSCE NA ZRZUT EKRANOWY DLA: 3. Ekran potwierdzenia zapisu] ---

4. Atomowy licznik zgłoszeń i bezpieczny zapis plików (flock)

Generowanie kolejnych numerów zgłoszeń z_001.txt, z_002.txt oparto o plik licznik.txt:

- Plik licznika otwierany jest w trybie dwukierunkowym fopen('licznik.txt', 'c+'),

- Blokada wyłączna flock($fp, LOCK_EX) nakładana jest natychmiast przed odczytem wartości,

- Po odczycie i inkrementacji następuje przewinięcie rewind($fp), wyczyszczenie bufora ftruncate($fp, 0), zapis nowej wartości z obsługą błędów zapisu, fflush($fp) oraz zwolnienie blokady.

- Plik zamówienia z_xxx.txt zapisywany jest za pomocą file_put_contents z flagą LOCK_EX w formacie określonym w instrukcji.

- Dodano plik .htaccess zabezpieczający pliki z danymi osobowymi (z_*.txt) i licznik przed bezpośrednim pobraniem przez protokół HTTP.

```
$fp = fopen($counter_file, "c+");
if (!$fp || !flock($fp, LOCK_EX)) {
    die("Blad serwera: niemozliwe nalozenie blokady na plik licznika.");
}
$raw_num = trim(stream_get_contents($fp));
$next_num = ($raw_num !== '') ? ((int)$raw_num + 1) : 1;
rewind($fp);
ftruncate($fp, 0);
fwrite($fp, (string)$next_num);
fflush($fp);
flock($fp, LOCK_UN);
fclose($fp);
file_put_contents($filename, $file_content, LOCK_EX);
```

--- [MIEJSCE NA ZRZUT EKRANOWY DLA: 4. Atomowy zapis z blokadą flock] ---

5. Monolityczna implementacja w jednym pliku a1.php

Wszystkie stany aplikacji (formularz, obsługa błędów, ekran potwierdzenia, zapis do plików, komunikaty końcowe i powrót) zostały zrealizowane w obrębie pojedynczego pliku a1.php.

--- [MIEJSCE NA ZRZUT EKRANOWY DLA: 5. Monolityczna implementacja a1.php] ---

Wnioski i podsumowanie

Aplikacja a1.php zapewnia rygorystyczne bezpieczeństwo danych wejściowych, całkowitą eliminację stanu wyścigu przy generowaniu numerów zamówień oraz poprawną architekturę sesyjną.